AI Logs

1. Proposal

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT to help narrow down my project idea about swearing and toxic comments into a concrete research question.	My original idea was too broad, and I wasn't sure how to turn it into a realistic machine learning experiment for the course project.	Formulated the core research question on whether swear words degrade classifier performance, and organized the proposal into 5 clear sections (Why, Who, Related Work, What, Evaluation).
I asked ChatGPT for relevant papers on how swear words affect toxicity classifiers.	I didn't know the literature in this area and needed some academic references.	Read and cited Pamungkas et al. (2020) and HateCheck (Röttger et al., 2021) in the Related Work section.
I asked ChatGPT for recommendations on good, publicly available toxicity datasets.	I needed an English benchmark dataset with human labels that was accessible to work with.	Selected the **Kaggle Jigsaw (2018)** Wikipedia toxic comments dataset.
I asked ChatGPT how previous studies operationalized swear words as an experimental variable and how orthographic variations (like leetspeak) affect dictionary lookups.	I wanted to follow standard methodology in the literature to identify swear words deterministically, while understanding what limitations to acknowledge in the proposal.	Adopted the **NoSwearing** dictionary following Pamungkas et al. (2020) to isolate profanity presence, and noted the limitation that exact lexical matching misses obfuscated spellings like *"f*ck"* or *"sh1t"* in `proposal.md`.
I asked ChatGPT to recommend a traditional machine learning baseline and explain why small encoder transformers (like DistilBERT or DeBERTa-v3-small) are suited for studying contextual swearing.	I needed an interpretable linear baseline and wanted to justify why contextual representations are theoretically necessary when words change meaning based on surrounding context.	Selected **TF-IDF with Logistic Regression** as the baseline and shortlisted small encoder-only transformers in the proposal's methodology in `proposal.md`.

2. Setup & Preprocessing

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT how to combine the 6 different toxic columns in the	The dataset has multiple labels like `toxic`, `severe_toxic`, `insult`,	Wrote a short function using `.any(axis=1).astype(int)` to create

Prompt(s)	Why?	Incorporated as...
Jigsaw dataset into a single 0 or 1 label in pandas.	etc., but my project only focuses on whether a comment is toxic or not.	the unified `is_toxic` target column in `main.py`.
I asked ChatGPT for an efficient way to search for hundreds of swear words across 160,000 comments without taking forever or matching parts of normal words.	Looping over the wordlist in Python was extremely slow, and simple string matching was wrongly flagging words like "ass" inside "class".	Compiled a single regex pattern using `re.escape` with word boundaries `\b` so it checks the entire wordlist in a single pass in `main.py`.
I asked ChatGPT how to write regex rules to clean up Wikipedia formatting (like `== Headers ==` and talk page indents), URLs, and elongated words like "sooooo".	Wikipedia comments have lots of markup, IP addresses, and stretched words that don't add useful signal and could mess up tokenization.	Built the `clean_data` function in `main.py` to strip wiki markup, replace URLs/IPs with tokens, and collapse repeated letters.
I asked ChatGPT how to save the cleaned data so I wouldn't have to re-run the cleaning steps every time I restarted the notebook.	Running regex over 160k comments takes several minutes, and saving to CSV was slow and sometimes messed up formatting.	Saved the cleaned train, validation, and test sets as Parquet files using `df.to_parquet()`, so they reload in seconds in `main.py`.

3. Slicing & Baseline Evaluation

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT how to use `train_test_split` to stratify across both `is_toxic` and `has_swear` at the same time.	Only ~10% of comments are toxic and ~8% have swear words, so a regular split could easily leave one set with too few swear words.	Passed both columns `stratify=train_df[["is_toxic", "has_swear"]]` so the proportion of toxic and swearing comments stays the same in train and validation (`main.py`).
I asked ChatGPT how to split the test set into 4 groups: toxic with swear words, toxic without, non-toxic with, and non-toxic without.	I needed to test the models on each group separately to see if they make more mistakes when swearing and toxicity don't match.	Created a `slice` column in the test set to separate the comments into the 4 groups so I could calculate metrics on each one independently (`main.py`).

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT for advice on good settings for a TF-IDF + Logistic Regression baseline on imbalanced text.	I wanted a strong, proper baseline before trying neural networks, but wasn't sure about n-gram ranges, feature limits, or class weights.	Built a pipeline with bigrams, sublinear TF scaling, a 25,000 feature limit, and `class_weight="balanced"` to handle the 90/10 class imbalance in `main.py`.
I asked ChatGPT which threshold-free evaluation metric works best when evaluating models on heavily imbalanced text data (90/10).	On an imbalanced dataset, standard aggregate metrics can be overly optimistic by rewarding majority-class predictions, so I needed a metric focused strictly on the minority (toxic) class across all decision thresholds.	Used `average_precision_score` in my evaluation function to compute and report PR-AUC alongside F1 score in `main.py`.
I asked ChatGPT how to calculate False Positive Rate (FPR) and compare recall differences between the slices in Python.	Scikit-learn doesn't have a built-in FPR function, and I needed an easy way to measure the gap in performance between comments with and without swear words.	Wrote the `evaluate_model` helper in `main.py` to compute FPR (`FP / (FP + TN)`) and return ΔRecall and ΔFPR to measure the swear word bias directly.

4. Transformer Training (DeBERTa)

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT how to figure out a good `max_length` for tokenizing comments without wasting GPU memory.	I didn't want to guess 512 and run out of memory, or guess too low and chop off half of the sentences.	Tokenized a sample of 10,000 comments without truncation, checked the length percentiles, and found that 128 tokens covered about 82% of all comments (`main.py`).

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT why DeBERTa is considered better at understanding word context than standard BERT or RoBERTa.	I wanted to understand the technical justification for picking DeBERTa-v3 over older models for a project focused on context.	Learned about its disentangled attention mechanism, which keeps word meaning and relative position separate, helping it understand how words like "not" affect nearby swear words.
I asked ChatGPT how to write a `compute_metrics` function for the Hugging Face `Trainer` to track F1, Precision, and Recall during training.	By default the Trainer only reports training and validation loss, but I needed F1 score to pick the best model checkpoint.	Wrote a custom metrics callback in `main.py` that converts logits with argmax and returns accuracy, precision, recall, and F1 at the end of each epoch.
I asked ChatGPT how to use gradient accumulation to train with an effective batch size of 32 when my GPU could only fit 8 or 16 samples.	Small batch sizes can make training unstable, but increasing the batch size caused out-of-memory errors on the GPU.	Set `gradient_accumulation_steps = target_batch_size // batch_size` and enabled `bf16=is_cuda` on the server to keep training fast and stable (`main.py`).
I asked ChatGPT how to save the model's test predictions so I wouldn't have to re-run the transformer every time I worked on the notebook.	Running inference on the full test set takes a long time, and I needed the prediction probabilities to plot curves and check errors.	Saved the raw prediction logits as a `.npy` file, and used `torch.softmax` to get the probabilities whenever needed (`main.py`).

5. Evaluation & Error Analysis

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT how to plot a Precision-Recall curve in matplotlib comparing two models and including the baseline line.	I wanted a clear visual comparison between TF-IDF and DeBERTa that showed how much better they performed compared to random guessing.	Plotted PR curves for both models in `main.py` with a dashed horizontal line at the dataset's toxic prevalence rate (`y_true.mean()`).
I asked ChatGPT how to extract the top positive and negative word weights from	I wanted to see which specific words the baseline relied on	Pulled `feature_names` and `coef_[0]` into a dataframe and sorted them to inspect the 10

Prompt(s)	Why?	Incorporated as...
the TF-IDF + Logistic Regression pipeline.	most to decide if a comment was toxic or non-toxic.	strongest toxic and non-toxic words (`main.py`).
I asked ChatGPT for an easy way to filter and inspect specific failure cases (like harmless comments with swearing that the models got wrong).	I needed real comment examples for my report and presentation to show where TF-IDF failed because of swear words and where DeBERTa succeeded or failed.	Filtered the analysis dataframe by slice and model prediction in `main.py` to pull out examples of false alarms and swear-free insults.

6. Report Proofreading

Prompt(s)	Why?	Incorporated as...
I asked ChatGPT to proofread `doc/report/report.tex` for typos, grammar, and consistent wording.	I wanted to catch careless typos (like duplicated words and misspellings), keep the phrasing natural and direct, and make sure I used terms like "swear words" consistently across all sections.	Fixed typos and awkward phrasing, standardized references to swear words across the slice tables and text, and made section headers clearer in `doc/report/report.tex`.